Agent Postmortem Generator

Master Engineering Roadmap

Open-source infrastructure roadmap for a local-first, evidence-based AI agent debugging tool

Prepared as a pre-development engineering plan for a startup-grade repository

Attribute	Decision
Product category	Open-source developer tool / AI agent observability utility
Primary user	Engineers debugging failed agent runs
MVP posture	Local-first, deterministic, framework-agnostic, evidence-based
Repository tone	Production-grade infrastructure project, not a demo
Initial scope	Generic JSON trace input with a single canonical analysis pipeline

1. Executive Summary

Agent Postmortem Generator should become the default tool for explaining failed AI agent executions the way pytest became the default for testing and ruff became the default for linting. The immediate job is narrow: ingest a failed agent trace, reconstruct the execution narrative, identify the first meaningful fault, and produce a blameless incident report that engineers can trust without manually reading thousands of log lines.

The MVP must not try to solve every observability problem on day one. It should win on a single axis: fast, reproducible, local analysis of agent failures with strong evidence provenance. That means the first release needs one dependable pipeline, one canonical output format, and one crisp user promise.

2. Vision Statement

Build the standard debugging primitive for AI agents: a local tool that turns messy execution traces into clear, structured postmortems with enough rigor that an engineer can act on them immediately.

3. Product Goals

- Make failure analysis reproducible and deterministic on a developer laptop.
- Explain what failed, why it failed, how the failure propagated, and what to fix first.
- Preserve evidence so users can verify every claim in the report.
- Stay framework-agnostic by defining an adapter boundary early.
- Keep the first release small enough to maintain without a platform team.

4. User Personas

Persona	Needs	What Success Looks Like
Platform / infra engineer	Needs fast root-cause analysis for agent incidents	Can isolate the first bad step and patch it in minutes
Applied AI engineer	Needs to understand why an agent drifted or stopped	Receives a readable report instead of raw trace noise
Open-source maintainer	Needs a clean issue triage artifact	Can ask contributors for the same structured report
SRE / reliability engineer	Needs postmortem discipline for agent workflows	Can reuse the report format for incident review

5. Problems Being Solved

- Execution traces are verbose, inconsistent, and hard to interpret by hand.
- Different agent frameworks structure events differently, which makes debugging tribal and repetitive.
- Engineers often jump straight to LLM explanations without grounding the output in evidence.
- Failures propagate across retries, tools, and planner loops, obscuring the original fault.
- Post-incident reviews are usually written manually, so teams lose time and consistency.

6. MVP Definition

The MVP is a local CLI that accepts a single failed run in a canonical trace format, analyzes the sequence deterministically, and emits a structured markdown postmortem with citations to the underlying trace events.

The first release should prove the product value with minimal moving parts. Anything that cannot be explained, tested, or maintained by two or three engineers in the first six months is not MVP material.

7. Features Included in MVP

- CLI entrypoint with local file input and local file output.
- Canonical trace schema for a generic JSON event log.
- Trace parser and validation layer.
- Deterministic event normalization pipeline.
- Failure boundary detection and first-fault heuristics.
- Structured report generator in Markdown.
- Evidence links for every major assertion in the report.
- Minimal plugin interface for future adapters.
- High-quality README, examples, and quickstart docs.
- Automated tests for parser, analysis, and report generation.

8. Features Explicitly Excluded from MVP

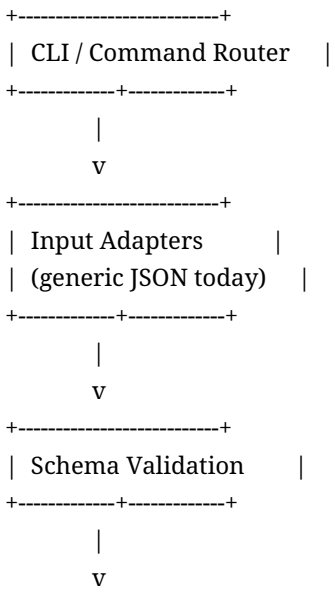
- OpenTelemetry ingestion
- Token usage analysis

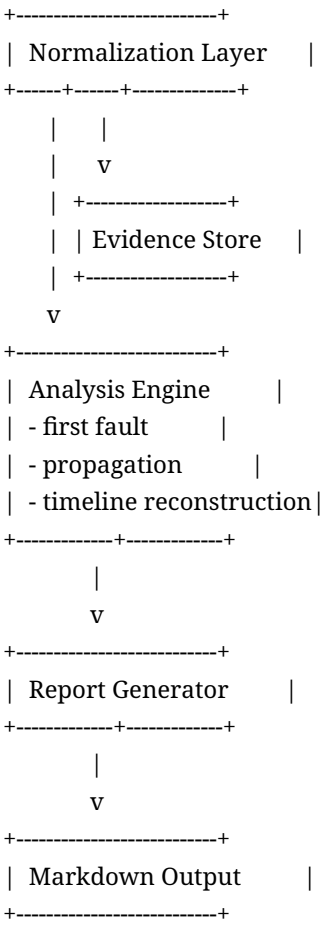
- Retry visualization
- Interactive execution graphs
- VS Code extension
- GitHub Action
- PDF export
- Drift detection
- Multi-run comparisons
- LLM-assisted report refinement
- Cloud hosted analysis
- Authentication, accounts, or billing
- Distributed processing
- Web UI

9. Milestone Breakdown

Milestone	Duration	Outcome	Exit Criteria
M0 - Architecture lock	Week 1	Canonical scope, schemas, boundaries	Spec approved and repo skeleton defined
M1 - Trace core	Weeks 2-3	Parser, validator, normalized event model	Can ingest one generic trace reliably
M2 - Analysis engine	Weeks 4-6	Failure detection and first-fault logic	Produces stable findings from known fixtures
M3 - Report layer	Weeks 7-8	Markdown postmortems with evidence links	Readable output matches expected template
M4 - Hardening	Weeks 9-10	Tests, fixtures, docs, packaging	Repo feels shippable and supportable
M5 - v1 release candidate	Weeks 11-12	Taggable release + demo dataset	A stranger can run it on their laptop

10. Dependency Graph Between Modules





Dependency principle: modules must flow one way. Adapters may depend on the schema, analysis may depend on normalized events, and reporting may depend on analysis outputs. Nothing in the lower layers should know about the CLI, the report format, or future UI ideas.

11. Recommended Development Order

1. Define the canonical trace schema and the report contract first.
2. Build the parser and validator before any analysis logic.
3. Implement normalization and evidence indexing immediately after parsing.
4. Add first-fault detection on top of normalized events.
5. Produce a deterministic markdown report before considering any formatting polish.
6. Add fixtures for real failure patterns, then harden with tests.
7. Only after the core loop is stable, add adapter abstractions for future frameworks.

12. Estimated Complexity Per Module

Module	Complexity	Reason
CLI / packaging	Low	Standard distribution work
Generic JSON parser	Medium	Schema ambiguity and malformed input handling
Normalization layer	High	Trace diversity creates edge cases
Evidence store	Medium	Requires disciplined event referencing

Failure boundary detection	High	Needs practical heuristics with deterministic output
Report generator	Medium	Template discipline and stable formatting
Tests / fixtures	High	Must cover real-world trace weirdness
Future adapters	Medium to High	Framework-specific semantics will vary

13. Risk Assessment

Risk	Impact	Likelihood	Mitigation
Trace formats are too inconsistent	High	High	Start with one canonical schema and force adapters to normalize early
Analysis becomes heuristic soup	High	Medium	Keep deterministic rules, require evidence, and log decision paths
Output is technically correct but not trusted	High	Medium	Expose provenance, cite source events, and keep wording precise
Scope creep into observability platform territory	High	High	Draw a hard line around v1 and defer dashboards and cloud ingestion
Few realistic fixtures to test against	Medium	High	Build a trace corpus from real incidents as soon as possible
Maintainers cannot support the project	Medium	Medium	Prioritize small module boundaries and exhaustive docs

14. Technical Debt Strategy

- No hidden magic in the analysis engine. Every rule should be explainable and testable.
- Prefer explicit schemas over loose dictionaries in internal boundaries.
- Keep adapter-specific quirks isolated from the canonical model.
- Avoid premature optimization until trace volumes justify it.
- Document every heuristic with a rationale and an expected failure mode.
- Treat report language as product surface, not as throwaway string assembly.

15. Release Strategy

Ship a small, opinionated v1 rather than a broad beta. The first public release should be useful on day one, even if it only supports the generic trace path.

Stage	Scope	Objective
Alpha	Internal repo only	Validate the schema, analysis,

		and report contract
Beta	Public repository with sample traces	Collect feedback from real users and frameworks
v1.0	Stable CLI and docs	Declare the first supported release line

16. Open Source Strategy

- Use a contributor-friendly repo with a single obvious entrypoint.
- Keep issues labeled by adapter, analysis, docs, and good first task.
- Maintain a strict compatibility policy for the canonical schema.
- Write design notes for major architectural decisions before implementation.
- Prefer deterministic fixtures over synthetic benchmark claims.
- Accept contributions only through clear module boundaries and tests.

17. Repository Organization Strategy

```
agent-postmortem-generator/
├── src/
│   ├── core/            # canonical model, validation, normalization
│   ├── analysis/        # first-fault logic, propagation, evidence selection
│   ├── report/          # markdown rendering and templates
│   ├── adapters/        # framework-specific input adapters
│   └── cli/             # command entrypoint
├── tests/
│   ├── fixtures/
│   └── unit/
├── docs/
│   ├── architecture/
│   └── examples/
├── examples/
└── pyproject.toml
```

Repository principle: the folder structure should make the architecture obvious without requiring a README scavenger hunt. If new contributors cannot orient themselves in under a minute, the repo is too clever.

18. Documentation Strategy

- README should answer what the tool does, how to run it, and what it does not do.
- Add an architecture doc that explains the canonical trace model and evidence flow.
- Publish a trace schema reference with examples and edge cases.
- Include a report format guide so contributors understand the expected output.
- Create a failure-corpus gallery with representative inputs and outputs.
- Document adapter contracts before adding the first non-generic adapter.

19. Demo Strategy

The demo should be embarrassingly simple and reproducible: one sample failed trace, one command, one generated postmortem. No browser, no sign-in, no orchestration theatre.

Demo Element	Purpose
Sample trace	Shows a realistic failure with retries, tool calls, and eventual crash
CLI command	Proves the product is local-first and fast
Generated postmortem	Demonstrates clarity, evidence, and blameless tone
Before/after comparison	Shows the time saved over raw log inspection

20. Success Metrics

Metric	Target	Why it matters
Time to first useful report	< 30 seconds on a laptop	Confirms local-first utility
Fixture pass rate	100% on maintained corpus	Proves determinism
Evidence coverage	Every major claim cites source events	Builds trust
User comprehension	Engineers can identify the first fault quickly	Measures practical value
Repository clarity	New contributor can find the core path in minutes	Supports open-source adoption
Release readiness	Single-command install and run	Reduces friction

21. Future Roadmap (Post-v1)

- OpenTelemetry ingestion for standard observability pipelines.
- Token usage and cost analysis.
- Retry visualization and richer propagation graphs.
- Interactive execution graphs for humans who insist on clicking things.
- VS Code extension for in-editor debugging.
- GitHub Action for automatic postmortems on CI failures.
- PDF export for incident review packages.
- Drift detection across repeated runs.
- Multi-run comparisons for regression analysis.
- LLM-assisted refinement as an optional layer after deterministic analysis.

22. Final Engineering Recommendations

- Lock the canonical trace schema before writing fancy heuristics.
- Keep the first release boring on purpose; boring is stable, and stable is valuable.
- Optimize for trust, not for novelty.
- Build the evidence model as a first-class system, not as an afterthought.
- Treat the repo like infrastructure software from day one: disciplined modules, explicit contracts, and no magic behavior.
- Reject any feature that cannot be defended as essential to the first useful report.